

Python开发音乐盒



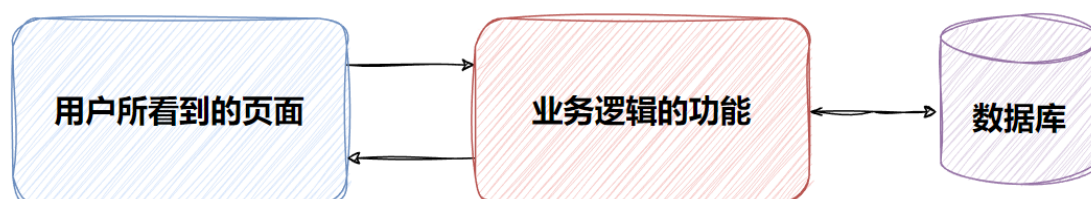
实现功能

模拟网易云音乐，实现本地音乐盒

- 登录功能
- 窗口显示
- 加载本地音乐
- 建立播放列表
- 音乐播放
- 删除播放列表音乐

实现结构

用户所看到的页面 + 业务逻辑的功能 + 数据库



无论 网页程序、小程序、微信公共号、手机APP，基本都逻辑都会是上图的样子

目的

- 练习数据库SQL的使用
- 数据库与应用的结合使用

需求分析-建立数据库表



飙升榜 歌曲

网易云音乐 最近更新：6月13日

播放全部 收藏(404万) 分享(1万) 下载全部

歌曲：99 播放：55亿

简介：云音乐中每天热度上升最快的100首单曲，每日更新。

歌曲列表 评论(218430) 收藏者

操作	音乐标题	飙升率	歌手	专辑	时长
01	你消失后 (Hi-Res)	718%	姜云升 / 刘美麟	你消失后	04:33
02	诀爱 尽 (LIVE版) (Hi-Res)	581%	盛宇D-SHINE / 詹雯婷	中国说唱巅峰对决2023 第六期加更	03:56
03	向云端 (Hi-Res)	326%	小霞 / 海洋Bo	向云端	04:11
04	七月七日晴 (SQ)	292%	王贰浪	七月七日晴	03:47
05	冬青 (Hi-Res)	215%	30年前, 50年后	冬青	03:19
06	You Ain't Right (Hi-Res) (MV)	210%	张靓颖	You Ain't Right	03:33
07	普鲁斯特时刻 (Hi-Res)	171%	范世錡	普鲁斯特	03:21
08	我都明白 (说唱版) (Hi-Res)	164%	Sapling / 费米Frieme / 范税Liu	我都明白 (说唱版)	03:30
09	一直很安静 (Live版) (Hi-Res)	141%	毛不易	剧好听的歌 第1期	04:07

数据库创建

```

1 # 创建数据库
2 CREATE DATABASE music_db
3     DEFAULT CHARACTER SET = 'utf8mb4';
  
```

切换数据库

```
1 # 切换数据库
2 use music_db;
```

创建用户表

```
1 # 创建用户表
2 CREATE TABLE t_user (
3     id int(11) PRIMARY KEY AUTO_INCREMENT,
4     uname varchar(32),
5     password varchar(32),
6     PRIMARY KEY (id)
7 );
```

创建音乐列表

```
1 CREATE TABLE t_music (
2     id int(11) PRIMARY KEY AUTO_INCREMENT,
3     name varchar(32),
4     singer varchar(32),
5     path varchar(128)
6 );
```

创建播放列表

```
1 CREATE TABLE t_play_list (
2     id int(11) PRIMARY KEY AUTO_INCREMENT,
3     u_id int(11),
4     m_id int(11),
5     FOREIGN KEY (u_id) REFERENCES t_user (id),
6     FOREIGN KEY (m_id) REFERENCES t_music (id)
7 );
```

实现登录功能

初始化用户数据

```
1 insert into t_user values('bz','123');
```

逻辑实现

```
1 from music_tool import DBUtil
2
3 class MusicSerice:
4     def login(self,uname:str,passwd:str):
5         # 查询用户的sql
6         sql = 'select id,uname from t_user
7 where uname=%s and passwd=%s'
8         # 执行查询
9         user =
10 DBUtil().query_one(sql,uname,passwd)
11 # 判断返回结果,如果user空,登录失败
12 if user:
13     print('登录成功')
14     return True
15 else:
16     print('登录失败')
17     return False
18
19 if __name__ == '__main__':
20     MusicSerice().login('bz','1234')
```

可能出现的错误1:

ModuleNotFoundError: No module named 'pymysql'

解决办法:

在cmd中输入

```
1 | pip install pymysql==1.0.3
```

可能出现的错误2:

AttributeError: 'DBUtil' object has no attribute 'qery_one'.
Did you mean: 'query_one'?

解决方案:

拼写错误, 一般意思就是我们函数名写的不对, 修改正确即可

实现主窗口的显示

```
1 import tkinter
2 from PlayerService import MyService
3
4 class Playerwindows:
5     def __init__(self, ser):
6         self.service = ser
7         top = tkinter.Tk()
8         # 创建播放按钮
9         btn1 = tkinter.Button(top, text="播
放")
10        # 创建导入音乐按钮
11        btn2 = tkinter.Button(top, text="导入
音乐")
12        btn3 = tkinter.Button(top, text="删
除")
```

```
13         # 创建播放列表
14         self.listbox = tkinter.Listbox(top)
15         # 设置播放按钮在第一行，第一列显示
16         btn1.grid(row=0, column=0, padx=5,
17                    pady=5)
18         # 设置播放按钮在第一行，第三列显示
19         btn2.grid(row=0, column=2, padx=5,
20                    pady=5)
21         btn3.grid(row=0, column=4, padx=5,
22                    pady=5)
23         # 设置播放列表在第二行，第一列显示
24         self.listbox.grid(row=1, column=0,
25                            padx=5, pady=5, columnspan=5)
26         # 进入消息循环
27         top.mainloop()
```

实现Python播放音乐

```
1 import pygame
2 import time
3 # 初始化-加载音乐的混合器
4 pygame.mixer.init()
5 # 加载音乐
6 pygame.mixer.music.load('D:\code\python\mysql_code\music_demo\music\爱的废墟.mp3')
7 # 播放音乐
8 pygame.mixer.music.play()
9
10 time.sleep(10)
```

可能出现的错误:

ModuleNotFoundError: No module named 'pygame'

解决办法:

在cmd中输入

```
1 | pip install pygame==2.4.0
```

导入音乐_UI设置事件

```
1 import tkinter
2 from tkinter.filedialog import
  askopenfilenames
3
4 from service import MusicService
5
6 class Playerwindow:
7     def __init__(self):
8         top = tkinter.Tk()
9         # 增加按钮
10        btn1 = tkinter.Button(top, text='播
  放')
11        btn2 = tkinter.Button(top, text='导
  入音乐')
12        btn3 = tkinter.Button(top, text='删
  除')
13        # 指定 按钮的位置
14        btn1.grid(row=0, column=0, padx=5,
  pady=5)
```

```
15         btn2.grid(row=0, column=2, padx=5,
16         pady=5)
17         btn3.grid(row=0, column=4, padx=5,
18         pady=5)
19         # 增加播放列表
20         listbox = tkinter.Listbox(top)
21         listbox.grid(row=1, column=0,
22         columnspan=5, padx=5, pady=5)
23
24         # 设置btn2的事件处理函数
25         btn2.bind('<Button-1>',
26         self.import_music)
27
28         # 进入消息循环
29         top.mainloop()
30
31         def import_music(self, event):
32             print('导入音乐')
33             # 弹出文件选择框
34             filenames =
35             askopenfilenames(filetype=[('mp3', '.mp3'),
36             ('flac', '.flac')])
37             print(filenames)
38
39         if __name__ == '__main__':
40             uname = input('请输入用户名: ')
41             passwd = input('请输入密码: ')
42             # 创建服务对象
43             ms = MusicService()
44             # 调用登录功能, 判断是否登录成功
45             if ms.login(uname, passwd):
46                 print('登录成功, 欢迎使用')
```

```

41         pw = PlayerWindow()
42     else:
43         print('登录失败')
44
45
46

```

导入音乐_服务设置

```

1  class MyService:
2      def add_music(self,files:tuple[str]) ->
None:
3          '''
4              增加音乐功能
5              :param files: 音乐文件路径，默认是多个，
里面的元素是字符串
6              :return: None
7              '''
8              # 编写sql
9              insert_music_sql = 'insert into
t_music (name,path) values (%s,%s)'
10             # 遍历数据
11             for f in files:
12                 print(f)
13                 # 根据路径名称获取音乐名
14                 name =
f[f.rfind('/')+1:f.rfind('.')]
15                 # 创建工具对象
16                 db = DBUtil()
17                 # 执行sql
18
db.execute_dml(insert_music_sql,name,f)

```

```

19
20 if __name__ == '__main__':
21     ser = MusicService()
22     # ser.login('admin','123456')
23     # ser.login('bz','123')
24
25     ser.add_music(('d:/aaa/1.mp3', 'd:/aaa/2.mp3'
26 ))

```

维护播放列表_用户和音乐关系

```

1 # service.py
2 from tools import DBUtil
3
4 class MusicService:
5     def __init__(self):
6         self.user = None
7     def login(self,uname:str,passwd:str) ->
8 bool:
9         '''
10         登录功能
11         :param uname: 用户名
12         :param passwd: 密码
13         :return: 登录成功返回True, 失败返回False
14         '''
15         # 编写sql
16         sql = 'select id,uname from t_user
17         where uname=%s and passwd=%s'
18         # 创建工具对象
19         db = DBUtil()

```

```

18         # 执行sql
19         user =
db.query_one(sql,uname,passwd)
20         # 判断是否登录成功, 判断user是否有值, 有值
        返回True, 否则返回False
21         if user:
22             # print('登录成功')
23             # 记录用户的信息
24             self.user = user
25             return True
26         else:
27             # print('登录失败')
28             return False
29
30     def add_music(self,files:tuple[str]) ->
None:
31         '''
32         增加音乐功能
33         :param files: 音乐文件路径, 默认是多个,
        里面的元素是字符串
34         :return: None
35         '''
36         # 编写sql
37         insert_music_sql = 'insert into
t_music (name,path) values (%s,%s)'
38         # 遍历数据
39         for f in files:
40             print(f)
41             # 根据路径名称获取音乐名
42             name =
f[f.rfind('/')+1:f.rfind('.')]
43             # 创建工具对象

```

```

44         db = DBUtil()
45         # 执行sql
46         mid =
db.execute_dml_back_id(insert_music_sql,name
,f)
47         # 编写用户和音乐关系的sql
48         insert_play_music_sql = 'insert
into t_play_list (u_id,m_id) values (%s,%s)'
49         # 创建工具对象
50         db = DBUtil()
51         # 执行sql
52
db.execute_dml(insert_play_music_sql,self.u
ser[0],mid)

```

```

1 # tools.py
2 def execute_dml_back_id(self,sql,*args):
3     '''
4     可以执行dml语句，用于数据的增加
5     '''
6     try:
7         # 执行SQL
8         self.cursor.execute(sql,args)
9         # 获取添加的id
10        id = self.con.insert_id()
11        # 提交事务
12        self.con.commit()
13        # 返回id
14        return id
15    except Exception as e:
16        print(e)

```

```
17         if self.con:
18             self.con.rollback()
19         finally:
20             self.close()
```

```
1 # ui.py
2     def import_music(self, event):
3         print('导入音乐')
4         # 弹出文件选择框
5         filenames =
askopenfilenames(filetype=[('mp3', '.mp3'),
('flac', '.flac')])
6
7         # 调用增加音乐功能
8         ms.add_music(filenames)
```

显示播放列表

```
1 # service.py
2     def find_user_music(self) -> list[str]:
3         # 编写sql
4         sql = 'select m.name from
5 t_play_list p left join t_music m on p.m_id
6 = m.id where u_id = %s'
7         # 创建工具对象
8         db = DBUtil()
9         # 执行sql
10        m_list =
db.query_all(sql, self.user[0])
11        # 返回结果
12        return m_list
```

```
1 # ui.py
2 class Playerwindow:
3     def __init__(self):
4         top = tkinter.Tk()
5         # 增加按钮
6         btn1 = tkinter.Button(top, text='播
7 放')
8         btn2 = tkinter.Button(top, text='导
9 入音乐')
10        btn3 = tkinter.Button(top, text='删
11 除')
12        # 指定 按钮的位置
13        btn1.grid(row=0, column=0, padx=5,
14 pady=5)
15        btn2.grid(row=0, column=2, padx=5,
16 pady=5)
```

```
12         btn3.grid(row=0, column=4, padx=5,
13         pady=5)
14         # 增加播放列表
15         self.listbox = tkinter.Listbox(top)
16         self.listbox.grid(row=1, column=0,
17         columnspan=5, padx=5, pady=5)
18
19         # 获取播放列表
20         self.load_music()
21
22         # 设置btn2的事件处理函数
23         btn2.bind('<ButtonRelease-1>',
24         self.import_music)
25         btn1.bind('<ButtonRelease-1>',
26         self.play_music)
27
28         # 进入消息循环
29         top.mainloop()
30
31         def import_music(self, event):
32             print('导入音乐')
33             # 弹出文件选择框
34             filenames =
35             askopenfilenames(filetype=[('mp3', '.mp3'),
36             ('flac', '.flac')])
37             # 调用增加音乐功能
38             ms.add_music(filenames)
39             # 获取播放列表
40             self.load_music()
41
42         def load_music(self):
43             # 调用获取播放列表功能
```

```

38         m_list = ms.find_user_music()
39         # 重新加载
40         for m in m_list:
41             self.listbox.insert(0,m[0])

```

播放音乐

```

1 # select m.id,name,path from t_music m RIGHT
  join t_play_list p on m.id = p.m_id where
  name='爱的废墟' and u_id = 1

```

```

1 # ui.py
2 class Playerwindow:
3     def play_music(self, event):
4         # 获取当前列表中选中的音乐
5         index = self.listbox.curselection()
6         # 根据索引获取音乐的名称
7         music_name = self.listbox.get(index)
8         print(music_name)
9         # 将音乐名称传递给服务对象
10        ms.play_music(music_name)

```

```

1 # service.py
2 class MusicService:
3     def play_music(self, music_name:str):
4         '''

```

```
5         播放音乐
6         :param music_name: 音乐名称
7         '''
8         # 获取音乐的路径path
9         sql = 'select m.path from
t_play_list p left join t_music m on p.m_id
= m.id where u_id = %s and m.name= %s'
10        # 执行sql
11        path =
DBUtil().query_one(sql,self.user[0],music_na
me)[0]
12        # 调用播放器播放音乐
13        # 初始化-加载音乐的混合器
14        pygame.mixer.init()
15        # 加载音乐
16        pygame.mixer.music.load(path)
17        # 播放音乐
18        pygame.mixer.music.play()
```

修复显示播放列表

```
1 # ui.py
2 class Playerwindow:
3
4     def load_music(self):
5         # 调用获取播放列表功能
6         m_list = ms.find_user_music()
7         # 清空列表
8         self.listbox.delete(0,tkinter.END)
9         # 重新加载
10        for m in m_list:
11            self.listbox.insert(0,m[0])
```

删除播放列表音乐

```
1 # ui.py
2 class Playerwindow:
3     def delete_music(self, event):
4         # 删除用户指定的音乐
5         index = self.listbox.curselection()
6         # 根据索引获取音乐的名称
7         music_name = self.listbox.get(index)
8         # 调用删除音乐功能
9         ms.delete_music(music_name)
10        # 重新加载播放列表
11        self.load_music()
```

```
1 # service.py
2     def delete_music(self, music_name:str):
3         '''
```

```
4         删除音乐
5         :param music_name: 音乐名称
6         '''
7         # 编写sql,查询要删除的音乐的id
8         get_id_sql = 'select m.id from
t_play_list p left join t_music m on p.m_id
= m.id where u_id = %s and m.name=%s'
9         # 执行sql
10        rs =
DBUtil().query_one(get_id_sql,self.user[0],m
usic_name)[0]
11
12        # 编写sql,删除播放列表
13        delte_play_list_sql = 'delete from
t_play_list where m_id = %s and u_id = %s'
14
DBUtil().execute_dml(delte_play_list_sql,rs
,self.user[0])
15        # 编写sql,删除音乐表
16        delete_music_sql = 'delete from
t_music where id = %s'
17
DBUtil().execute_dml(delete_music_sql,rs)
```